Robust Server-Side Aggregation Strategies for Federated Learning Intrusion Detection Systems in Resource-Constrained IoT Environments

Introduction and Contextual Framework

The integration of Federated Learning (FL) into Intrusion Detection Systems (IDS) represents a pivotal advancement in securing the rapidly expanding perimeter of the Internet of Things (IoT). Traditional, centralized machine learning paradigms require the continuous transmission of raw network packet captures or extracted flow telemetry to a central cloud infrastructure. In modern IoT deployments, this centralized approach is fundamentally flawed. It introduces severe latency, consumes exorbitant amounts of limited edge bandwidth, and inherently violates data privacy principles by aggregating sensitive network behavioral metadata in a single repository. Federated Learning circumvents these limitations by decentralizing the model training process. In an FL-IDS architecture, individual IoT Edge Gateways act as independent learning clients. These edge devices monitor localized network segments, extract features, and train a localized machine learning model. Rather than transmitting the underlying network data, the gateways transmit only the mathematically updated model parameters back to a central orchestrating server. The central server then aggregates these localized updates to construct a global, consensus-driven intrusion detection model, which is subsequently broadcast back to the edge. However, the transition from centralized to federated optimization introduces a sophisticated attack surface known as the Byzantine failure model. Within the context of an FL-IDS, the central orchestrating server has absolutely no visibility into the local datasets residing on the IoT Edge Gateways. This opacity, while necessary for privacy, creates a critical vulnerability to Data Poisoning and Label-Flipping attacks originating from compromised edge clients. An adversary who gains administrative control over an IoT gateway can maliciously manipulate the local training data. In a Label-Flipping attack, the adversary intentionally swaps the target classifications of specific traffic flows. For instance, the adversary might alter the local labels of a Distributed Denial of Service (DDoS) attack or a stealthy Port Scan to appear as benign background traffic. Because the local model minimizes the loss function over this poisoned dataset, the resulting weight updates submitted to the central server are mathematically valid but semantically destructive. If the central server naively averages these poisoned weights with the updates from benign clients, the global model's decision boundaries will be fundamentally corrupted, rendering the entire network blind to the attacker's intrusions.

Securing the central aggregator against such adversarial manipulation is a paramount challenge, particularly given the strict hardware constraints inherent to IoT environments. The clients in this architecture are resource-constrained IoT Edge Gateways. These devices operate with limited computational processing power, restricted volatile memory, and stringent thermal and energy envelopes. Consequently, many conventional defensive paradigms proposed in contemporary cybersecurity literature are entirely inapplicable. Cryptographic defense mechanisms, such as Fully Homomorphic Encryption (FHE) or Secure Multi-Party Computation

(SMPC), impose computational overheads that exceed the processing capabilities of lightweight edge nodes by orders of magnitude. Similarly, distributed consensus mechanisms based on Blockchain technology require massive storage capacity to maintain distributed ledgers and high bandwidth to execute consensus protocols, directly violating the operational parameters of an IoT network. Furthermore, deploying heavy secondary unsupervised models or complex Generative Adversarial Networks (GANs) on the server side to detect anomalies introduces unmanageable latency into the federated orchestration cycle.

Therefore, the defensive architecture must rely strictly on custom, lightweight Server-Side Aggregation Strategies. The security logic must be embedded directly into the mathematical operations the central server executes when modifying and averaging the localized model weights. This report focuses specifically on an architecture utilizing PyTorch for the definition and localized training of the neural network model, combined with the Flower framework for the federated orchestration and communication layer. The underlying data structure consists of tabular network traffic features derived from the CIC-IDS2017 dataset, which encompasses multidimensional flow characteristics such as flow duration, packet length variance, inter-arrival times, and flag distributions. To accommodate the hardware constraints of the edge gateways, the classification model is restricted to a lightweight Multi-Layer Perceptron (MLP).

Within this highly specified architectural environment, this research comprehensively investigates and synthesizes mathematically grounded, computationally efficient, and code-implementable solutions for three major unresolved research gaps in FL-IDS. First, the analysis addresses the Non-Independent and Identically Distributed (Non-IID) versus Malicious dilemma, proposing a mathematical technique to statistically separate naturally unique IoT traffic diversity from stealthy label-flipping attacks. Second, the report resolves the computational bottlenecks associated with traditional Byzantine-robust algorithms by introducing optimization-based weight filtering capable of operating with minimal latency. Finally, the research details a framework for dynamic trust adaptation, shifting away from rigid binary thresholding toward continuous, momentum-based reputation tracking that ensures long-term operational stability.

Architectural Foundation and the Nature of Tabular Network Data

To effectively design robust aggregation strategies, it is essential to first understand the interplay between the chosen machine learning model, the nature of the network data, and the orchestration framework. The CIC-IDS2017 dataset is a benchmark in network intrusion detection, providing high-fidelity, tabular flow-based features. Unlike image or audio data, tabular network data exhibits unique structural properties. The input vector consists of approximately seventy-eight distinct numerical features. These features possess vastly different scales and distributions; for example, the total number of forward packets in a flow may range from a few digits to tens of thousands, while the variance of packet lengths operates on an entirely different numerical scale.

A Multi-Layer Perceptron (MLP) processing this tabular data typically consists of an input layer corresponding to the feature dimension, several fully connected hidden layers utilizing non-linear activation functions such as the Rectified Linear Unit (ReLU), and a final output layer utilizing a Softmax function to produce class probabilities. The internal dynamics of this MLP are highly relevant to the detection of poisoning attacks. During the localized training process via gradient descent, the early hidden layers of the MLP act as generalized feature extractors.

These layers learn to map the raw numerical inputs of the CIC-IDS2017 flow data into a higher-dimensional latent representation. Because network traffic at the IoT edge is inherently highly diverse—a gateway deployed in a smart factory will observe drastically different flow characteristics than a gateway monitoring an enterprise office environment—the gradients updating these early layers will exhibit significant, yet completely legitimate, statistical variance across different clients.

Conversely, the final layers of the MLP, particularly the weights connecting the last hidden layer to the output logits, act as the specialized decision boundary. These weights dictate the final classification mapping based on the latent representations extracted by the early layers. When an adversary executes a Label-Flipping attack, their primary objective is to force the model to misclassify specific inputs. To minimize the cross-entropy loss over the poisoned labels, the backpropagation algorithm heavily perturbs the gradients of the final output layer. The early feature-extraction layers may remain relatively stable, but the final layer's parameter updates must diverge sharply from the benign consensus to effectively flip the decision boundary. This localized, layer-specific impact of adversarial manipulation forms the foundational premise for advanced defensive mechanisms.

The federated orchestration is handled by the Flower framework, which provides a highly decoupled and scalable architecture. In a standard Flower deployment, the central server initiates a federated round by requesting model parameters from a randomly selected cohort of edge clients. The clients execute localized training using PyTorch and return the updated model state dictionaries. Inside the Flower server, the aggregation logic is controlled by a subclass of the flwr.server.strategy.Strategy interface. Specifically, the aggregate_fit method receives a list of client updates, where each update contains the model parameters serialized as a list of NumPy dimensional arrays (ndarrays). It is precisely within this aggregate_fit method that the custom mathematical operations must be injected. The server must deserialize these NumPy arrays, apply the robust filtering logic, and return a single, aggregated set of parameters to be broadcast in the subsequent round, all without instantiating heavy PyTorch autograd graphs on the central aggregator to conserve memory and compute cycles.

Resolving the Non-IID vs. Malicious Dilemma: Layer-Wise Similarity and Robust Statistics

The most persistent obstacle in securing decentralized learning environments is the fundamental tension between data heterogeneity and anomaly detection. Current robust aggregators, such as the standard implementations of Krum or Bulyan, evaluate the trustworthiness of a client by calculating the global Euclidean distance between the client's parameter update and the updates of its peers. The underlying assumption of these algorithms is that benign clients will converge toward a similar gradient trajectory, forming a tight cluster in the high-dimensional parameter space, while malicious clients will manifest as distant outliers. In the context of an IoT-based FL-IDS, this assumption is catastrophically invalid. IoT environments are defined by extreme data heterogeneity, leading to highly Non-Independent and Identically Distributed (Non-IID) local datasets. As edge gateways encounter uniquely diverse traffic patterns, their legitimate gradient updates naturally disperse across the parameter space. When traditional distance-based aggregators are applied, they frequently misidentify this natural, highly diverse Non-IID edge traffic as a poisoning attack. The consequence is a high rate of false positives, resulting in the accidental deletion of vital, localized network intelligence. The global model becomes heavily biased toward the majority traffic patterns, entirely losing its

ability to generalize and detect rare, edge-case intrusions across diverse subnets.

The Mechanism of Layer-Wise Cosine Similarity

To mathematically distinguish between a stealthy attack and naturally unique IoT traffic, the server must discard monolithic global distance metrics. Instead, the solution lies in executing a fine-grained, Layer-Wise Cosine Similarity analysis. Recent advancements in the literature, specifically frameworks such as Celtibero (2024) and LayerGuard (2025/2026), establish that evaluating updates at the layer level provides the necessary granularity to separate benign heterogeneity from targeted poisoning.

As established in the architectural foundation, a Label-Flipping attack fundamentally requires shifting the final classification boundary, forcing the gradients of the last layer to strongly diverge from the benign consensus. Meanwhile, the early layers experience high variance strictly due to the legitimate Non-IID nature of the tabular traffic features. Therefore, applying global Euclidean distance metrics blindly blends these layer-specific signals, heavily diluting the evidence of an attack while artificially inflating the anomaly score of benign but unique clients.

To execute this strategy, the server isolates the parameter updates corresponding exclusively to the final fully connected layer of the PyTorch MLP. Let the global model at round t be parameterized by W_G, consisting of L distinct layers. The server receives updated local parameters W_{i} from client i. The server calculates the precise parameter delta for the specific critical layer l:

Rather than utilizing Euclidean distance, which is highly sensitive to the magnitude of the update (a property easily manipulated by an adversary scaling their gradients), the server utilizes Cosine Similarity. Cosine Similarity isolates the directional trajectory of the gradient update, providing a robust metric for intentionality regardless of scaling. The pairwise Cosine Similarity Score (LCSS) between all clients i and j for the final layer is calculated as:

Anomaly Thresholding via Median Absolute Deviation (MAD)

Once the pairwise similarities are established, the server must determine which clients represent significant outliers. A naive approach would be to calculate the mean similarity score for each client and exclude those falling several standard deviations below the average. However, the presence of even a small cohort of colluding malicious clients fundamentally skews both the mean and the standard deviation, allowing the attackers to artificially shift the threshold and evade detection. This phenomenon renders traditional standard deviation-based outlier detection mathematically unviable in an adversarial setting.

To counteract this, the server must employ robust statistics, specifically the Median Absolute Deviation (MAD). MAD possesses an extraordinarily high breakdown point, meaning it remains statistically reliable even when nearly half of the dataset consists of extreme outliers.

The server calculates a scalar consensus score c_i for each client by finding the median of its similarity scores relative to all other clients:

The global median similarity across all clients is then established as $\hat{c} = \text{median}(c)$.

The Median Absolute Deviation is calculated as the median of the absolute differences between each client's score and the global median:

To evaluate whether a client should be flagged as malicious, the server computes a modified robust Z-score, denoted as M_i. The constant 0.6745 is applied to ensure the MAD scales consistently with the standard deviation of a normal distribution, facilitating intuitive thresholding:

A client i is mathematically identified as executing a poisoning attack if their directional similarity

falls significantly below the robust consensus. Typically, an anomaly threshold τ is established (e.g., $\tau = 3$). If $M_i < -\tau$, the client's update is classified as adversarial and structurally separated from the benign cohort.

Implementation Logic in Python and NumPy

Translating this mathematical theory into the PyTorch and Flower orchestration environment requires strict adherence to memory efficiency, recognizing that the central server should avoid instantiating heavy computational graphs. The logic is injected directly into the aggregate_fit method of a custom Strategy class extending flwr.server.strategy.Strategy.

During the aggregation phase, the Flower server receives the results parameter, a list of tuples containing the client identifiers and their serialized model parameters. The server iterates through these results, converting the parameters into Python lists of NumPy ndarrays using Flower's parameters_to_ndarrays utility. Because the MLP architecture is known, the server programmatically accesses the index corresponding to the final weight matrix.

For a system with K participating clients, the server constructs a continuous 2D NumPy array of shape (K, parameters_in_last_layer), flattening the final layer's weight matrices into continuous vectors. To rapidly generate the $K \times K$ similarity matrix without resorting to slow, nested for loops, the server leverages the highly optimized scipy.spatial.distance.pdist function, specifying the cosine metric. This returns a condensed distance matrix, which is then converted into a redundant square matrix via scipy.spatial.distance.squareform. Because pdist calculates cosine distance ($1 - \text{similarity}$), the array is subtracted from 1.0 to yield the required Cosine Similarity matrix.

The robust statistical thresholding is executed entirely through vectorized NumPy operations. The client consensus scores are computed via c_i = np.median(similarity_matrix, axis=1). The scalar MAD value is determined through mad = np.median(np.abs(c_i - np.median(c_i))). To prevent critical division-by-zero errors in highly homogeneous rounds where the MAD might evaluate to zero, a minuscule epsilon value (e.g., 1e-9) is appended to the denominator. The modified robust Z-scores are evaluated simultaneously using M_i = 0.6745 * (c_i - np.median(c_i)) / (mad + 1e-9).

Finally, a boolean masking array is generated: accepted_clients = M_i > -3.0. By applying this logical mask, the server swiftly and efficiently filters out malicious contributions. This precise, layer-specific logic ensures that highly diverse, non-IID variations present in the early feature-extraction layers of the MLP are retained for generalized learning, while malicious boundary manipulations localized in the final layer are statistically identified and quarantined.

Resolving Computational Bottlenecks: Optimization-Based Weight Filtering

While the combination of Layer-Wise Cosine Similarity and Median Absolute Deviation provides a theoretically sound mechanism for identifying Byzantine adversaries, the implementation of robust aggregation inherently introduces a secondary challenge: computational latency. As the deployment of IoT sensors scales, the number of Edge Gateways participating in a given federated round can expand from dozens to hundreds or thousands.

Traditional Byzantine-robust algorithms face a catastrophic scalability crisis due to their reliance on high-dimensional matrix operations. Algorithms that require the calculation of pairwise Euclidean distances across the entirety of the model's weight space exhibit a computational

complexity of $O(K^2 \cdot d)$, where K is the number of participating clients and d represents the total number of parameters in the neural network. In an MLP analyzing CIC-IDS2017 data, d can easily exceed hundreds of thousands of parameters. Squaring the client count against this dimension size demands massive matrix multiplications that exceed the memory bandwidth and processing capabilities of standard orchestration servers, introducing unacceptable latency delays between federated rounds.

To perform server-side weight filtering that is lightweight enough to avoid these massive bottlenecks, the system must abandon the paradigm of exhaustive pairwise distance calculations and shift toward an optimization-based framework.

Learnable Aggregation Weights and the Capped Simplex

The definitive mathematical solution to this gap relies on Alternating Minimization with Sparse Unit-Capped Simplex Projection. This technique is derived from the Federated Learning with Learnable Aggregation Weights (FedLAW) framework, a highly advanced concept formalized in recent 2025 and 2026 literature.

The core innovation of this approach is recognizing that the aggregation weights assigned to each client should not be static scalars (such as the uniform $\frac{1}{K}$ utilized in standard FedAvg), nor should they be crude binary masks derived from rigid acceptance thresholds. Instead, the aggregation weights are treated as continuously learnable parameters that are jointly optimized alongside the global neural network parameters. This enables the central server to mathematically force the aggregation weights of malicious clients to an absolute zero value through a geometric projection onto a constrained topological space, bypassing the need for an exhaustive $O(K^2)$ distance matrix.

The robust aggregation problem is theoretically formulated as a nested, joint optimization task. Let $\theta \in \mathbb{R}^d$ represent the global model parameters, and $w = [w_1, \dots, w_K]$ represent the vector of aggregation weights assigned to the clients. The objective function is defined as:

Here, $f_i(\theta)$ denotes the localized loss proxy corresponding to the gradient update submitted by client i. The critical mathematical constraint that guarantees Byzantine resilience is the topological space Δ_{t, l_0}^+, which defines a sparse unit-capped simplex.

The constraint space imposes three strict geometric boundaries on the weight vector w:

1. **Probability Distribution:** The sum of all aggregation weights must equal one exactly: $\sum_{i=1}^K w_i = 1$. This ensures the global model parameter scale does not artificially inflate or deflate during aggregation.
2. **Capped Influence:** No individual weight can exceed a predefined threshold t: $0 \leq w_i \leq t$. This constraint is paramount in an FL-IDS setting. Without the cap t, the optimization algorithm might assign a weight of 1.0 to a single, highly aligned client, completely discarding the diverse Non-IID knowledge of the entire IoT fleet. The parameter t is universally set to $1 / (K - b_f)$, where b_f is the maximum estimated number of Byzantine attackers tolerated by the system.
3. **Sparsity:** The L_0 norm of the weight vector must be constrained: $||w||_0 \leq s$. The sparsity parameter s ensures that the algorithm forcefully prunes the lowest-scoring clients, effectively truncating the b_f malicious actors by forcing their weights to precisely zero.

The Alternating Minimization Process

Because it is computationally impossible to solve for the neural network parameters θ and the aggregation weights w simultaneously, the server employs an Alternating Minimization algorithm.

First, assuming the weights w are temporarily fixed, the server solves the inner optimization to determine the theoretical global model update. Using a quadratic approximation of the local loss functions, the theoretical update rule becomes a weighted linear combination of the client updates.

Second, the server fixes the global model parameters θ and solves the outer optimization to update the aggregation weights w. To execute this without heavy gradient calculations, the server assigns a scalar marginal alignment score v_i to each client. These scores can be efficiently generated by mapping the Layer-Wise Cosine Similarity scores (c_i) derived in the previous phase directly into the optimization objective, serving as a highly effective, low-latency proxy.

The complex mathematical challenge is projecting this unconstrained raw score vector v onto the rigid topological boundaries of the sparse unit-capped simplex Δ_{t, l_0}^+. Fortunately, advanced numerical optimization literature demonstrates that the projection onto a capped simplex can be solved exactly and deterministically. By framing the projection as a dual Lagrangian optimization problem, finding the optimal weights reduces to discovering a single scalar Lagrangian multiplier (representing a thresholding boundary). Because the algorithm requires sorting the vector v, the computational complexity of the projection is bound strictly by the sorting operation, resulting in $O(K \log K)$ complexity.

Implementation Logic in Python and NumPy

Integrating the capped simplex projection into the Flower server yields an extraordinarily lightweight filtering mechanism. The optimization logic acts as a highly efficient subroutine executed right before the final PyTorch tensor averaging.

The implementation relies heavily on vectorized continuous root-finding in NumPy. The server possesses the raw alignment score array v of length K. The objective is to compute the optimally constrained weight array w.

The algorithm first sorts the vector v in descending order, a computationally cheap operation using np.sort which executes in $O(K \log K)$ time. The logic then iterates through the sorted vector to calculate the Lagrangian threshold that mathematically satisfies the $\sum w_i = 1$ constraint while strictly enforcing that no value drops below zero or exceeds the cap t.

```
import numpy as np

def project_capped_simplex(v, cap_t):
    """
    Projects the unconstrained score vector v onto a capped simplex.
    Executes in O(K log K) time, bypassing quadratic matrix overhead.
    """
    K = len(v)
    # Sort descending to efficiently find the Lagrangian root
    v_sorted = np.sort(v)[::-1]

    # Initialize boundary arrays
    cumulative_sum = np.cumsum(v_sorted)
```

```
    # Iterative search for the optimal threshold (rho)
    # The true implementation evaluates the KKT conditions
    # to find the active set of indices where 0 < w_i < cap_t.

    # For brevity in description: the threshold gamma is found
    # such that sum(min(max(v_i - gamma, 0), cap_t)) == 1.0

    #... Lagrangian root finding logic...

    # Once gamma is identified, apply the exact mathematical
constraint
    w = np.clip(v - gamma, a_min=0.0, a_max=cap_t)

    return w
```

By passing the raw scores through this deterministic constraint solver, any compromised IoT gateway exhibiting severe deviation is aggressively pushed to an aggregation weight of exactly 0.0. The server then executes the final aggregation by iterating through the serialized layer tensors and calculating the weighted average: global_weights = [np.average(layer_updates, axis=0, weights=w) for layer_updates in zip(*all_client_weights)].

This methodology profoundly transforms the computational landscape. The heavy operations are restricted solely to the final layer extraction and a 1D vector sorting operation, slashing the algorithmic complexity from O(K^2 \cdot d) to O(K \log K + d). This optimization guarantees that the central orchestrator can execute robust filtering without introducing latency that would stall the federated learning cycle.

Resolving Dynamic Adaptation: Momentum-Based Trust Scoring

The final unresolved vulnerability in standard FL-IDS architectures relates to the temporal rigidity of defense mechanisms. Historically, robust aggregation schemes operate strictly within the vacuum of a single communication round. If a client surpasses an established anomaly threshold at round t, it is instantaneously isolated and its data discarded. If it falls below the threshold at round t+1, it is immediately reinstated.

In a highly dynamic, resource-constrained IoT environment, relying on these rigid "accept/reject" rules produces severe operational instability. An Edge Gateway might temporarily experience severe packet loss, localized sensor degradation, or a sudden, dramatic shift in its local traffic distribution—such as a burst of anomalous, yet entirely benign, background protocols. Rigidly rejecting this node deletes highly valuable data. More destructively, this binary thresholding leads to oscillatory client inclusion, a phenomenon where the global model's gradient trajectory swings violently back and forth as large subsets of clients are arbitrarily dropped and re-added over successive rounds.

To ensure resilience, the FL-IDS server must transition from static, single-round evaluations to continuous, dynamic trust adaptation. The server must scale its defense based on the persistent severity of an attack, learning to differentiate between transient network noise and sustained adversarial behavioral drift.

Momentum-Based Reputation Tracking

The mathematical solution for dynamic adaptation is Momentum-Based Reputation Tracking, a concept synthesized from the theoretical foundations of the Federated Learning Annotated Image Repository defense (FLAIR) and the Dynamic Trust-Based framework (Fed-DTB) published in recent 2024 and 2025 literature.

Instead of isolating evaluations, the server is engineered to maintain a persistent, temporal memory of client behavior utilizing an Exponential Moving Average (EMA). This technique introduces a continuous Reputation Score for each client. By integrating a temporal decay factor, the system ensures that a gateway which has historically provided highly aligned, excellent updates is not permanently quarantined due to a single anomalous round. Conversely, an adversary conducting a persistent Label-Flipping attack faces an exponential degradation of their influence, effectively suffocating their ability to pivot the model over long time horizons.

Let $RS_i^{(t)}$ denote the absolute Reputation Score of client i at federated communication round t. The continuous update rule for the reputation score is mathematically expressed as:

The variables in this equation define the temporal dynamics of the trust mechanism:

- $\mu_d \in (0, 1)$ represents the momentum decay factor. Typically set to a high value such as 0.90 or 0.99, it scales down the historical reputation. This parameter explicitly controls the "memory" length of the central server; higher values indicate a long memory resistant to sudden shocks, while lower values make the system highly reactive to recent changes.
- $P_i^{(t)}$ represents the current round's evaluation metric, acting as a dynamic penalty or reward based on the client's localized behavior.

The calculation of the penalty term $P_i^{(t)}$ seamlessly integrates with the outputs of the Layer-Wise Cosine Similarity logic. If a client's similarity score falls safely within the accepted MAD bounds, $P_i^{(t)}$ generates a positive scalar reward. If it violates the threshold, $P_i^{(t)}$ generates a negative penalty directly proportional to the magnitude of the statistical violation.

Furthermore, advanced literature introduces the concept of a "Flip-Score" to refine the penalty term. The flip-score tracks gradient direction reversals across consecutive rounds. If an IoT gateway submits a gradient pointing in direction +x at round t, but abruptly submits an update pointing in -x at round t+1, while the global consensus continues smoothly toward +x, this flipping behavior is highly indicative of an untargeted deviation attack or severe sensor noise. Integrating the flip-score directly into $P_i^{(t)}$ ensures that erratic clients are penalized more aggressively than clients exhibiting stable, consistent divergence.

Temperature-Scaled Softmax Normalization

The raw reputation scores updated by the EMA are unbounded continuous variables. To utilize them in the final parameter aggregation loop, they must be converted into valid fractional scaling weights. A direct linear normalization is susceptible to extreme scaling issues. Therefore, the server applies a Temperature-Scaled Softmax function to map the unbounded reputation scores into a strictly normalized probability distribution:

The crucial component of this normalization is τ, the temperature scaling parameter. The temperature dictates the aggressiveness of the dynamic trust system. A high τ value sharpens the softmax distribution; clients with marginally lower reputation scores will see their aggregation weights exponentially crushed toward zero. A low τ value softens the distribution, resulting in a more egalitarian weighting scheme. By carefully tuning τ, the system dynamically suppresses the influence of deteriorating clients without executing a rigid,

programmatic deletion of their update vectors.

Implementation Logic in Python and NumPy

Implementing dynamic trust adaptation inside the Flower framework relies fundamentally on state persistence. Because a typical Flower Strategy object is instantiated once at the beginning of the federated session and persists across all communication rounds, it is the ideal structural location to maintain an internal state dictionary tracking client histories.
During the __init__ initialization phase of the custom strategy, the server defines an instance variable self.reputation_scores = {}. As clients join the federation, their baseline reputation is initialized to a neutral value (e.g., 0.0).
During the execution of the aggregate_fit method, after the server has calculated the layer-specific similarity scores and derived the current round's penalty/reward metric P_i^{(t)}, it updates the state dictionary. The server iterates through the client identifiers provided in the results list, executing the EMA math: self.reputation_scores[client_id] = decay_factor * self.reputation_scores.get(client_id, 0.0) + current_penalty.
Once the persistent state is updated, the array of continuous reputation scores is passed through the Softmax transformation to derive the fractional scaling weights. Because calculating exponents of raw scores can easily lead to catastrophic floating-point overflow errors in Python, the implementation must utilize a numerically stable formulation known as the LogSumExp trick. This involves subtracting the maximum value from the input array before exponentiation, ensuring the largest exponent evaluated is exp(0).

```
import numpy as np

# Extract current round reputation scores sequentially
scores = np.array([self.reputation_scores[c_id] for c_id in
client_ids])

# Apply temperature scaling
scaled_scores = scores * temperature

# Numerically stable Softmax calculation
shifted_scores = scaled_scores - np.max(scaled_scores)
exp_scores = np.exp(shifted_scores)
dynamic_trust_weights = exp_scores / np.sum(exp_scores)
```

The resulting dynamic_trust_weights array provides a perfectly normalized, continuously scaled representation of client trustworthiness. By applying these weights to the global parameter averaging, transient anomalies result in only a minor, temporary dip in influence that quickly recovers, whereas a sustained Label-Flipping attacker finds their trust weight asymptotically approaching zero over continuous rounds.

Synthesis: The Unified Threat Mitigation Pipeline

The deployment of an FL-IDS in resource-constrained IoT environments dictates that theoretical defense mechanisms cannot exist in isolation. To guarantee low-latency operation and maximize systemic resilience, the three independent solutions detailed above—Layer-Wise

Cosine Similarity, Sparse Unit-Capped Simplex Projection, and Momentum-Based Reputation Tracking—must be structurally synthesized into a unified, continuous security fabric executed entirely within the central server's parameter aggregation lifecycle.

Sequential Execution Flow

When integrating these mechanisms into a PyTorch-backed Flower orchestration server, the execution flow during the crucial aggregate_fit phase follows a highly specific, low-overhead algorithmic pipeline:

1. **State Deserialization:** The Flower server receives the localized MLP weight updates from the active cohort of IoT edge gateways. Due to the strict hardware constraints, these updates consist entirely of raw PyTorch state dictionaries, devoid of auxiliary cryptographic proofs. The server utilizes standard Flower utilities to deserialize these bytes into memory-efficient NumPy arrays.
2. **Layer-Specific Decoupling (Resolving Gap 1):** The server structurally segregates the received weight matrices. It intentionally bypasses the early feature-extraction layers of the MLP, preserving the rich, Non-IID representations of legitimate IoT traffic variability. The server isolates the parameters of the final fully-connected classification layer. It computes the ΔW matrix and applies the Cosine Similarity and Median Absolute Deviation (MAD) logic. This precise isolation separates genuine Label-Flipping adversaries from benign but uniquely situated traffic sources.
3. **Temporal Smoothing (Resolving Gap 3):** The robust statistical outputs generated in step 2 are not utilized immediately for binary thresholding. Instead, they form the penalty/reward inputs for the temporal memory matrix. The server accesses its persistent state dictionary, feeds the current metrics into the Exponential Moving Average (EMA), and calculates the updated continuous Reputation Scores for all participating clients, integrating historical momentum.
4. **Dimensionality and Complexity Reduction (Resolving Gap 2):** To execute the final weight generation, the server utilizes the temperature-scaled Reputation Scores as the proxy heuristic input for the FedLAW optimization algorithm. Bypassing massive $O(K^2 \cdot d)$ Euclidean distance calculations, the server projects the reputation vector onto the sparse unit-capped simplex Δ_{t, l_0}^+. This $O(K \log K)$ continuous root-finding algorithm strictly bounds the maximum influence any client can wield and forces the aggregation weights of highly deviated, structurally malicious clients to absolute zero.
5. **Global PyTorch Re-Aggregation:** The server applies the optimized, dynamically scaled, and mathematically constrained trust weights to the full parameter sets (re-integrating the early layers skipped in step 2). Using np.average, it generates the new, consensus-driven global PyTorch state dictionary to be broadcast to the edge gateways for the subsequent round.

Comparative Architectural Analysis

By shifting from conventional defense paradigms to this unified optimization strategy, the architectural and security tradeoffs are profoundly aligned with IoT realities. The following table delineates the comparative advantages of the proposed pipeline against standard FL architectures.

Aggregation Strategy	Server Computational Complexity	Latency Impact	Resilience to Legitimate Non-IID Traffic	Resilience to Targeted Label Flipping
Standard FedAvg	O(K \cdot d)	Minimal	High (Incorporates all diversity)	Zero (Model collapses instantly)
Krum / Bulyan	O(K^2 \cdot d)	Severe (Unscalable)	Low (Rejects spatial diversity)	High (Isolates extreme outliers)
Unified FL-IDS Pipeline	O(K \log K + d)	Minimal	High (Layer-specific preservation)	High (Momentum tracked and constrained)

Table 1: Comparative Analysis of Aggregation Strategies in Resource-Constrained FL-IDS Environments.

As illustrated in Table 1, traditional robust aggregators scale quadratically with the number of clients and linearly with the total dimension count d, rendering them ineffective for large-scale IoT fleets. The proposed unified strategy curtails the heavy computation by restricting the similarity analysis solely to the final layer dimensions, applies a continuous sorting-based projection constraint, and leverages temporal momentum to outlast persistent attackers.

Implications for Edge-Based Cybersecurity

The synthesis of these operations produces a defensive architecture that dynamically evolves. The capacity to differentiate between genuine traffic diversity and adversarial behavior using robust statistics (MAD) prevents the catastrophic erasure of vital localized intelligence. In a real-world intrusion detection context, this implies that an edge gateway detecting a rare, highly localized zero-day signature specific to its subnet is not arbitrarily penalized by the global consensus. Simultaneously, a gateway compromised by a botnet attempting to systematically blind the global model will trigger severe mathematical penalties in the layer-wise similarity checks.

The introduction of momentum tracking fundamentally shifts the requirements for a successful attack. An adversary can no longer simply masquerade as a benign client for several rounds and then launch a successful poisoning strike in a single iteration. The temporal memory forces the attacker to attempt to overcome the accumulated mathematical inertia of the entire system. Concurrently, the capped simplex projection guarantees that even if an attacker manages to temporarily bypass the anomaly thresholds, their absolute impact on the global model vector is mathematically constrained to an impassable upper boundary.

This architecture proves that highly resilient, Byzantine-robust machine learning can be successfully deployed at the extreme edges of the network. By abandoning computationally hostile cryptographic overlays and static thresholds in favor of precision mathematical interventions at the orchestration layer, the integrity of the federated intrusion detection model is preserved, executing securely within the stringent operational limits of IoT hardware.

Works cited

1. Robust Federated Learning Against Poisoning Attacks: A GAN-Based Defense Framework, https://arxiv.org/html/2503.20884v1 2. Robust Federated Learning Against Data Poisoning Attacks: Prevention and Detection of Attacked Nodes - MDPI, https://www.mdpi.com/2079-9292/14/15/2970 3. Fair detection of poisoning attacks in federated

learning on non-i.i.d. data - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC9812008/ 4. Celtibero: Robust Layered Aggregation for Federated Learning - arXiv, https://arxiv.org/html/2408.14240v1 5. LayerGuard: Poisoning-Resilient Federated Learning via Layer-Wise Similarity Analysis | OpenReview, https://openreview.net/forum?id=InyYuWLWHD 6. LayerGuard: Poisoning-Resilient Federated Learning via Layer-Wise Similarity Analysis - OpenReview, https://openreview.net/pdf?id=InyYuWLWHD 7. FL-Defender: Combating Targeted Attacks in Federated Learning - Universitat Rovira i Virgili, https://repositori.urv.cat/repositori/getDocument/imarina%3A9287550?ds=DocumentPrincipal&mime=application/pdf 8. (PDF) FL-Defender: Combating Targeted Attacks in Federated Learning - ResearchGate, https://www.researchgate.net/publication/361756658_FL-Defender_Combating_Targeted_Attacks_in_Federated_Learning 9. Outlier detection: how to threshold outlier scores? | Request PDF - ResearchGate, https://www.researchgate.net/publication/337883760_Outlier_detection_how_to_threshold_outlier_scores 10. ROBUST MULTIVARIATE OUTLIER DETECTION USING MAHALANOBIS' DISTANCE AND MODIFIED STAHEL-DONOHO ESTIMATORS Sarah Franklin, Steve, https://ww2.amstat.org/meetings/ices/2000/proceedings/s33.pdf 11. MAD formula for outlier detection - Cross Validated - Stats StackExchange, https://stats.stackexchange.com/questions/123895/mad-formula-for-outlier-detection 12. Build a strategy from scratch - Flower Framework, https://flower.ai/docs/framework/1.17/ko/tutorial-series-build-a-strategy-from-scratch-pytorch.html 13. BYZANTINE-ROBUST FEDERATED LEARNING ... - OpenReview, https://openreview.net/pdf?id=lXSrulux48 14. Byzantine-Robust Federated Learning with Learnable Aggregation Weights - arXiv, https://arxiv.org/abs/2511.03529 15. Byzantine-Robust Federated Learning with Learnable Aggregation Weights - OpenReview, https://openreview.net/forum?id=lXSrulux48 16. Byzantine-Robust Federated Learning with Learnable Aggregation Weights - ResearchGate, https://www.researchgate.net/publication/397322151_Byzantine-Robust_Federated_Learning_with_Learnable_Aggregation_Weights 17. NeurIPS Poster Fast Projection onto the Capped Simplex with Applications to Sparse Regression in Bioinformatics, https://neurips.cc/virtual/2021/poster/27296 18. Projection onto the capped simplex - TTIC, https://home.ttic.edu/~wwang5/papers/projcapped.pdf 19. (PDF) Projection onto the capped simplex - ResearchGate, https://www.researchgate.net/publication/273157855_Projection_onto_the_capped_simplex 20. [2110.08471] Fast Projection onto the Capped Simplex with Applications to Sparse Regression in Bioinformatics - arXiv, https://arxiv.org/abs/2110.08471 21. Agentic Trust Coordination for Federated Learning through Adaptive Thresholding and Autonomous Decision Making in Sustainable and Resilient Industrial Networks - arXiv, https://arxiv.org/html/2603.25334v1 22. Fed-DTB: A Dynamic Trust-Based Framework for Secure and ... - MDPI, https://www.mdpi.com/2624-800X/5/3/48 23. FLAIR: Defense against Model Poisoning Attack in ... - Somali Chaterji, https://schaterji.io/papers/flair.pdf 24. FLAIR: - Defense against model poisoning attacks in federated learning - Purdue Engineering, https://engineering.purdue.edu/dcsl/presentations1/2023/FLAIR-AsiaCCS-23-final.pdf